

Phase 1 Notes: Queue Fundamentals and the Queue ADT

1. Big Idea of This Phase

In Phase 1, we learn the basic idea of a **queue** before going deeply into code.

A queue is first a **rule of behavior**. After we understand the rule, we can later learn how to build it using arrays, linked lists, circular arrays, or other data structures.

The main rule of a queue is:

First In, First Out

This is called **FIFO**.

2. What Is a Queue?

A **queue** is a data structure that works like a real-life line.

Example: people waiting in line at a bank.

Front			Rear
Person A	Person B	Person C	

The person who entered the line first should be served first.

So if Person A came before Person B and Person C, Person A should leave first.

That is exactly how a queue works in programming.

3. FIFO: First In, First Out

FIFO means:

The first item inserted is the first item removed.

Example:

Inserted order: 10, 20, 30
Removed order: 10, 20, 30

The value 10 entered first, so 10 leaves first.

Then 20 leaves.

Then 30 leaves.

This is the most important rule of a queue.

4. Real-Life Examples of Queues

Queues are common in real life.

Examples:

People waiting in a line
Cars waiting at a toll booth
Print jobs waiting for a printer
Customers waiting for service
Tasks waiting to be processed by a computer

In all of these examples, the item that arrives first is usually handled first.

5. Front and Rear

A queue has two important ends:

Term	Meaning
Front	The side where deletion/removal happens
Rear	The side where insertion happens

A normal queue looks like this:

Deletion happens here



Front

[10, 20, 30]

Insertion happens here



Rear

So remember:

Insert at rear.
Delete from front.

6. Enqueue and Dequeue

The two most important queue operations are:

enqueue
dequeue

7. Enqueue

Enqueue means to insert a new value into the queue.

Insertion happens at the **rear**.

Example:

Queue before: 10, 20
Operation: enqueue(30)
Queue after: 10, 20, 30

The new value `30` joins at the rear of the queue.

8. Dequeue

Dequeue means to remove a value from the queue.

Removal happens from the **front**.

Example:

```
Queue before: 10, 20, 30
Operation: dequeue()
Queue after: 20, 30
```

The value `10` is removed because it was inserted first.

So `10` leaves first.

9. Queue vs Stack

Beginners often confuse queues and stacks.

They are different.

A **queue** follows FIFO:

```
First In, First Out
```

A **stack** follows LIFO:

```
Last In, First Out
```

Simple comparison:

Structure	Rule	Real-Life Example
Queue	FIFO	People standing in a line
Stack	LIFO	Plates stacked on top of each other

In a queue, the first person in line leaves first.

In a stack, the last plate placed on top is removed first.

10. What Is an ADT?

ADT means:

```
Abstract Data Type
```

In simple words, an ADT describes **what a data structure should do**, not exactly **how it is built inside memory**.

For a queue ADT, we care about rules such as:

```
Insert at rear.  
Delete from front.  
Follow FIFO.  
Check whether the queue is empty.  
Check whether the queue is full.
```

The internal implementation can be different.

A queue can be built using:

```
Array  
Linked list  
Circular array  
Two stacks
```

But no matter how we build it, it should still behave like a queue.

That means it should still follow FIFO.

11. Queue as a Logical Data Structure

A queue is called a **logical data structure** because the main idea is the rule, not the physical storage.

The logical rule is:

```
The first inserted item should be the first removed item.
```

The physical storage can be different.

For example, we may store queue elements using an array.

Or we may store them using linked nodes.

The storage method can change, but the queue behavior should remain the same.

12. Basic Queue Operations

A queue usually supports these operations:

Operation	Meaning
<code>enqueue()</code>	Insert a new element at the rear
<code>dequeue()</code>	Delete and return the element from the front
<code>isEmpty()</code>	Check whether the queue has no elements
<code>isFull()</code>	Check whether a fixed-size queue has no free space
<code>first()</code> or <code>front()</code>	Look at the oldest element without deleting it
<code>last()</code> or <code>rear()</code>	Look at the newest element without deleting it
<code>display()</code>	Show all elements currently in the queue

13. Time Complexity of Basic Queue Operations

In the two-pointer array design taught in this phase, most basic operations are fast.

Operation	Work Done	Time Complexity
<code>enqueue()</code>	Insert one element at rear	$O(1)$
<code>dequeue()</code>	Remove one element from front	$O(1)$
<code>isEmpty()</code>	Compare <code>front</code> and <code>rear</code>	$O(1)$
<code>isFull()</code>	Check whether <code>rear</code> reached the last index	$O(1)$
<code>first()</code>	Read front element	$O(1)$
<code>last()</code>	Read rear element	$O(1)$
<code>display()</code>	Print all queue elements	$O(n)$

$O(1)$ means the operation takes constant time.

It does not depend on how many elements are in the queue.

$O(n)$ means the operation depends on the number of elements.

Displaying all elements is $O(n)$ because every element must be printed.

14. Queue Structure in C/C++

In this phase, the queue is represented using a structure.

```
struct Queue {  
    int size;    // total capacity of the queue  
    int front;   // position used for deletion  
    int rear;    // position used for insertion  
    int *Q;      // array that stores the queue elements  
};
```

15. Meaning of Each Variable

Variable	Simple Meaning
size	Maximum number of values the queue can store
front	Tracks the deletion side of the queue
rear	Tracks the insertion side of the queue
Q	Pointer to the array where values are stored

Important point:

In this array version, `front` and `rear` are **index values**.

They are not actual memory addresses.

16. Starting State of the Queue

At the beginning, the queue is empty.

We write:

```
q.front = q.rear = -1;
```

This means:

```
front = -1  
rear = -1
```

Both are equal, so the queue is empty.

The empty condition is:

```
front == rear
```

If `front` and `rear` are equal, there is no valid element between them.

So:

```
front == rear means the queue is empty.
```

17. Important Mental Model for `front`

In this beginner-friendly design, `front` does not directly point to the first valid element.

Instead, `front` points to the position **before** the first valid element.

That means the first valid element is at:

```
front + 1
```

Example:

```
front = -1  
rear = 2  
Array values: [10, 20, 30]
```

The first valid element is at:

```
front + 1 = 0
```

So the first valid value is:


```
Q[0] = 10
```

18. Empty Condition

The queue is empty when:

```
front == rear
```

Example:

```
front = -1  
rear = -1
```

The queue is empty.

Another example:

```
front = 2  
rear = 2
```

The queue is also empty because there are no valid elements between `front` and `rear`.

19. Full Condition in This Simple Array Queue

The queue is full when `rear` reaches the last valid index.

The last valid index is:

```
size - 1
```

So the full condition is:

```
rear == size - 1
```

Example:

```
size = 5  
last index = 4
```

If:

```
rear = 4
```

then the queue is full in this simple linear array version.

20. isEmpty() Function

This function checks whether the queue is empty.

```
bool isEmpty(Queue *q) {  
    return q->front == q->rear;  
}
```

Simple meaning:

If front and rear are equal, the queue has no valid elements.

21. isFull() Function

This function checks whether the queue is full.

```
bool isFull(Queue *q) {  
    return q->rear == q->size - 1;  
}
```

Simple meaning:

If rear is at the last index, there is no more space at the rear.

22. Enqueue Logic Step by Step

To insert a value into the queue:

1. Check whether the queue is full.
2. If it is full, show an overflow/full message.
3. If it is not full, move `rear` one step forward.
4. Store the new value at `Q[rear]`.

Code:

```
void enqueue(Queue *q, int x) {  
    if (isFull(q)) {  
        cout << "Queue is full\n";  
    } else {  
        q->rear++;  
        q->Q[q->rear] = x;  
    }  
}
```

23. Enqueue Dry Run

Start:

```
front = -1  
rear = -1  
Queue is empty
```

Operation:

```
enqueue(10);
```

Steps:

```
rear moves from -1 to 0  
Q[0] = 10
```

Queue:

10

Now:

```
front = -1  
rear = 0
```

Operation:

```
enqueue(20);
```

Steps:

```
rear moves from 0 to 1  
Q[1] = 20
```

Queue:

10, 20

Now:

```
front = -1  
rear = 1
```

24. Dequeue Logic Step by Step

To remove a value from the queue:

1. Check whether the queue is empty.
2. If it is empty, show an underflow/empty message.
3. If it is not empty, move `front` one step forward.
4. Return the value at `Q[front]`.

Code:

```
int dequeue(Queue *q) {  
    int x = -1;
```

```
    if (isEmpty(q)) {  
        cout << "Queue is empty\n";  
    } else {  
        q->front++;  
        x = q->Q[q->front];  
    }  
  
    return x;  
}
```

25. Dequeue Dry Run

Suppose the queue contains:

10, 20, 30

The index positions are:

Q[0] = 10
Q[1] = 20
Q[2] = 30

Current pointer values:

front = -1
rear = 2

Now call:

dequeue();

Steps:

front moves from -1 to 0
x = Q[0]
x = 10

So 10 is returned and removed logically.

The queue now logically contains:

20, 30

Important idea:

The old value 10 may still physically exist in the array, but it is no longer part of the queue because front has moved past it.

This is called **logical deletion**.

26. Logical Deletion vs Physical Deletion

When we dequeue in this array queue, we usually do not erase the value from memory.

Instead, we move the front index.

Example:

```
Before dequeue:  
front = -1  
rear = 2  
Array = [10, 20, 30]
```

After one dequeue:

```
front = 0  
rear = 2  
Array may still look like [10, 20, 30]
```

But logically the queue is:

20, 30

Why?

Because the valid queue elements are between:

```
front + 1 and rear
```

So the valid range is now:

```
1 to 2
```

That gives:

```
Q[1] = 20  
Q[2] = 30
```

27. Dry Run with A, B, and C

Let us follow a full example using letters.

Starting State

```
front = -1  
rear = -1  
Queue is empty
```

Step 1: Enqueue A

Operation:

```
enqueue A
```

What happens:

```
rear moves from -1 to 0  
A is stored at index 0
```

Queue:

```
A
```

Pointers:

```
front = -1  
rear = 0
```

Step 2: Enqueue B

Operation:

```
enqueue B
```

What happens:

```
rear moves from 0 to 1  
B is stored at index 1
```

Queue:

```
A, B
```

Pointers:

```
front = -1  
rear = 1
```

Step 3: Enqueue C

Operation:

```
enqueue C
```

What happens:

```
rear moves from 1 to 2  
C is stored at index 2
```


Queue:

A, B, C

Pointers:

```
front = -1  
rear = 2
```

Step 4: Dequeue

Operation:

dequeue

What happens:

```
front moves from -1 to 0  
A is removed
```

Queue:

B, C

Pointers:

```
front = 0  
rear = 2
```

A entered first, so A leaves first.

That proves FIFO.

28. Dry Run Table

Step	Operation	Front Meaning	Rear Meaning	Logical Queue Content
1	Start	Before first element	Same as front	Empty
2	Enqueue A	Still before first element	Points at A	A
3	Enqueue B	Still before first element	Points at B	A, B
4	Enqueue C	Still before first element	Points at C	A, B, C
5	Dequeue	Moves to A position	Points at C	B, C

29. Complete Beginner Program

```
#include <iostream>
using namespace std;

struct Queue {
    int size;
    int front;
    int rear;
    int *Q;
};

bool isEmpty(Queue *q) {
    return q->front == q->rear;
}

bool isFull(Queue *q) {
    return q->rear == q->size - 1;
}

void enqueue(Queue *q, int x) {
    if (isFull(q)) {
        cout << "Queue is full\n";
    } else {
        q->rear++;
        q->Q[q->rear] = x;
    }
}

int dequeue(Queue *q) {
    int x = -1;
```

```

        if (isEmpty(q)) {
            cout << "Queue is empty\n";
        } else {
            q->front++;
            x = q->Q[q->front];
        }

        return x;
    }

int main() {
    Queue q;

    q.size = 5;
    q.front = q.rear = -1;
    q.Q = new int[q.size];

    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);

    cout << dequeue(&q) << " leaves first\n";
    cout << dequeue(&q) << " leaves second\n";

    delete[] q.Q;

    return 0;
}

```

30. Program Explanation

Step 1: Create a Queue Variable

```
Queue q;
```

This creates a queue object named `q`.

Step 2: Set the Queue Size

```
q.size = 5;
```

This means the queue can store up to 5 values in this simple array version.

Step 3: Make the Queue Empty

```
q.front = q.rear = -1;
```

This makes both `front` and `rear` equal to `-1`.

So the queue starts empty.

Step 4: Create the Array

```
q.Q = new int[q.size];
```

This creates an array of integers in heap memory.

The queue will store its values inside this array.

Step 5: Insert Values

```
enqueue(&q, 10);  
enqueue(&q, 20);  
enqueue(&q, 30);
```

The values enter in this order:

```
10, 20, 30
```

Step 6: Remove Values

```
cout << dequeue(&q) << " leaves first\n";  
cout << dequeue(&q) << " leaves second\n";
```

The values leave in this order:

```
10, 20
```

10 leaves first because it was inserted first.

Step 7: Free the Memory

```
delete[] q.Q;
```

The array was created using `new[]`, so it should be released using `delete[]`.

31. Program Output

```
10 leaves first
20 leaves second
```

This output proves the FIFO rule.

The queue inserted:

```
10, 20, 30
```

The first two removed values were:

```
10, 20
```

So the first inserted value left first.

32. Important Queue Rules to Remember

```
Queue follows FIFO.
FIFO means First In, First Out.
Insertion happens at the rear.
Deletion happens from the front.
enqueue means insert.
dequeue means delete/remove.
front == rear means the queue is empty.
rear == size - 1 means the queue is full in this simple linear array version.
front and rear are index values in the array version.
```

33. Overflow and Underflow

Two common queue problems are:

```
Overflow  
Underflow
```

34. Queue Overflow

Overflow happens when we try to insert into a full queue.

Example:

```
Queue size = 3  
Queue content = 10, 20, 30
```

Now if we try:

```
enqueue(40)
```

The queue has no space.

So this is overflow.

In code, we check this before insertion:

```
if (isFull(q)) {  
    cout << "Queue is full\n";  
}
```

35. Queue Underflow

Underflow happens when we try to delete from an empty queue.

Example:

```
Queue is empty
```

Now if we try:

```
dequeue()
```

There is nothing to remove.

So this is underflow.

In code, we check this before deletion:

```
if (isEmpty(q)) {  
    cout << "Queue is empty\n";  
}
```

36. Common Beginner Mistakes

Mistake 1: Confusing Queue and Stack

A queue is FIFO.

A stack is LIFO.

```
Queue: first item leaves first.  
Stack: last item leaves first.
```

Mistake 2: Deleting from the Rear

In a normal queue, deletion does not happen from the rear.

Deletion happens from the front.

```
Correct: delete from front  
Wrong: delete from rear in a normal queue
```

Mistake 3: Inserting at the Front

In a normal queue, insertion does not happen at the front.

Insertion happens at the rear.

Correct: insert at rear
Wrong: insert at front in a normal queue

Mistake 4: Forgetting to Check Full Before Enqueue

Before inserting, always check whether the queue is full.

```
if (isFull(q)) {  
    cout << "Queue is full\n";  
}
```

If we do not check, we may write outside the array.

That can cause errors.

Mistake 5: Forgetting to Check Empty Before Dequeue

Before deleting, always check whether the queue is empty.

```
if (isEmpty(q)) {  
    cout << "Queue is empty\n";  
}
```

If we do not check, we may try to remove something that does not exist.

Mistake 6: Thinking `front` and `rear` Are Always Address Pointers

In this array implementation, `front` and `rear` are not memory addresses.

They are index values.

Example:


```
front = 0  
rear = 2
```

This means positions in the array, not memory addresses.

Mistake 7: Forgetting That `front` Points Before the First Valid Element

In this design, the first valid element is at:

```
front + 1
```

So when displaying or reading the first valid element, do not start directly at `front`.

Start from:

```
front + 1
```

37. Phase 1 Summary

In Phase 1, we learned that a queue is a data structure that follows FIFO.

FIFO means the first inserted item is the first removed item.

A queue has two main ends:

```
front  
rear
```

Insertion happens at the rear.

Deletion happens from the front.

The main queue operations are:

```
enqueue  
dequeue  
isEmpty  
isFull
```

```
first/front element  
last/rear element  
display
```

We also learned the basic queue structure:

```
struct Queue {  
    int size;  
    int front;  
    int rear;  
    int *Q;  
};
```

We learned that:

```
front == rear means empty  
rear == size - 1 means full in this simple array version
```

We also saw a dry run showing that if **A**, **B**, and **C** enter the queue, then **A** leaves first.

That proves the FIFO rule.

38. Quick Review Questions

Question 1

If we enqueue these values:

```
5, 10, 15
```

Which value leaves first?

Answer:

```
5
```

Question 2

What operation inserts a value into a queue?

Answer:

enqueue

Question 3

What operation removes a value from a queue?

Answer:

dequeue

Question 4

Where does insertion happen in a normal queue?

Answer:

rear

Question 5

Where does deletion happen in a normal queue?

Answer:

front

Question 6

What does FIFO stand for?

Answer:

First In, First Out

Question 7

What condition means the queue is empty in this design?

Answer:

```
front == rear
```

Question 8

What condition means the queue is full in this simple linear array design?

Answer:

```
rear == size - 1
```

39. Final Takeaway

A queue is like a line.

The first value to enter is the first value to leave.

That one idea controls everything in Phase 1.

```
Queue = FIFO  
Insert at rear  
Delete from front
```

Once this is clear, the next phase becomes easier because we can focus on how to store the queue inside an array.